

MCP Servers & Custom Tool Integration

Prove one tool before adding the next

Session 10 of 19 | 3 hours

Agenda

Segment	Time
MCP request path and boundaries	8 min
Names, descriptions, and schemas	10 min
Stage - one tool walkthrough	15 min
Bad input and visible errors	10 min
Tests and MCP Inspector	9 min
Add tools and hand off to the lab	8 min

The MCP request path

```
Client
  ↓ tools/list
Tool catalog
  ↓ tools/call
Schema validation
  ↓
Handler
  ↓
Success or visible error
```

MCP standardizes discovery and calls. It does not approve the server or make its result correct.

Five boundaries to review

Boundary	Review question
Catalog	Which capabilities can the client discover?
Schema	Which arguments are accepted?
Handler	What work happens after a call?
Result	Can the caller distinguish failure from success?
Transport	Which process or endpoint carries the request?

Start with stdio and synthetic data. Keep the first review small.

Stage one exposes one tool

```
{  
  "name": "get_weather",  
  "description": "Return deterministic synthetic current weather for one city.",  
  "inputSchema": {  
    "type": "object",  
    "required": ["city"],  
    "additionalProperties": false  
  }  
}
```

The tool name, description, and schema are part of the interface.

Trace the good request

```
{  
  "name": "get_weather",  
  "arguments": {  
    "city": "Oslo"  
  }  
}
```

```
{  
  "city": "Oslo",  
  "temperature": 30,  
  "unit": "celsius",  
  "conditions": "rain"  
}
```

Deterministic synthetic output makes exact tests possible.

Trace the bad request

```
{
  "name": "get_weather",
  "arguments": {
    "city": "  "
  }
}
```

```
{
  "error": "city must be a non-empty string"
}
```

The MCP result sets `isError: true`. An invalid request must not look like an empty success response.

Schema and handler checks work together

Schema	Handler
Helps clients form valid calls	Protects direct and protocol calls
Documents required fields	Applies business limits
Rejects extra properties	Returns a stable error shape

Do not assume every caller reaches the handler through the same client-side check.

Test the handler before the client

```
cd lab/starter/custom-mcp  
npm ci  
npm test
```

The stage-one suite proves:

- one tool is exposed;
- good input returns exact data;
- an empty city returns a tool error.

Inspect the protocol path

```
npx --yes @modelcontextprotocol/inspector@2.5.0 --cli \  
node src/index.js --method tools/list
```

```
npx --yes @modelcontextprotocol/inspector@2.5.0 --cli \  
node src/index.js --method tools/call \  
--tool-name get_weather --tool-arg city=Oslo
```

Pin the Inspector version so recorded evidence remains reproducible.

Each check proves a different claim

Evidence	Claim
Unit test	Handler behavior is correct for known inputs
<code>tools/list</code>	The expected catalog is exposed
<code>tools/call</code>	The stdio protocol path reaches the handler
Copilot call	One approved client can discover and choose the tool

Keep the deterministic tests even when the client integration works.

Add the remaining tools

Tool	Contract
get_forecast	city ; days defaults to 3 and accepts 1–7
convert_temperature	finite value ; fixed source-unit enum

Unknown tools return a visible error. The first tool's tests must stay green.

Package the project, not dependencies

Track:

```
package.json  
package-lock.json  
src/  
test/
```

Do not track:

```
node_modules/
```

Use `npm ci` to reproduce the approved dependency tree.

Lab handoff

1. Run the one-tool starter tests.
2. Add one focused `get_weather` case.
3. Inspect good and bad calls.
4. Add `get_forecast`.
5. Add `convert_temperature`.
6. Run all five solution tests.
7. Record Inspector output or **not executed**.

The deliverable is a three-tool server with exact request, response, and failure evidence.